

省博物馆参观预约系统 · 前端开发设计

省博物馆参观预约系统前端开发设计

1. 文档目的

本文档用于指导省博物馆参观预约系统前端开发，明确前端技术栈、页面结构、路由设计、接口调用方式、页面与后端接口对应关系、前端校验规则、联调测试方案和开发顺序。

本文档依据：

- 项目设计开发流程.md
- 项目要求分析.md
- project-docs/需求拆分.md
- project-docs/系统设计文档初稿.md
- project-docs/数据库设计.md
- project-docs/后端接口设计.md
- 已完成的 Spring Boot 后端接口代码

当前后端接口已经按照 后端接口设计.md 全部实现并测试通过，因此前端可以直接基于后端 REST API 进行页面开发和联调。

2. 前端开发目标

前端需要完成一个可运行、可演示、可联调的省博物馆分时预约系统，主要目标包括：

- 游客可以查看场馆信息和公告通知。
- 游客可以实名登录。
- 游客可以查询可预约日期和时段。
- 游客可以提交预约并查看预约成功凭证。
- 游客可以根据身份证号查询自己的预约记录。
- 管理员可以登录后台。
- 管理员可以维护场馆信息。
- 管理员可以维护公告通知。
- 管理员可以维护预约活动。
- 管理员可以维护预约时段和名额。
- 管理员可以查询预约名单。
- 管理员可以导出预约名单 CSV。
- 管理员可以查看后台统计和系统状态。

前端不直接访问数据库，所有数据均通过 Axios 调用后端接口获得。

3. 前端技术栈

根据 项目设计开发流程.md 中的项目定位，前端技术栈采用：

Vue 3 + Axios + Bootstrap

建议具体版本与工具：

构建工具：Vite
前端框架：Vue 3
路由管理：Vue Router
HTTP 请求：Axios
UI 样式：Bootstrap 5
图表增强：可选 Chart.js
二维码展示：可选 qrcode.vue 或普通文本凭证

说明：

- Bootstrap 适合课程大作业，开发速度快，部署简单。
- Vue Router 用于游客端和管理员端页面切换。
- Axios 统一封装后端接口调用。
- 后端暂未引入正式 Token 鉴权，因此前端登录状态第一版可使用 localStorage 保存基础登录信息。

4. 前端项目目录结构设计

前端项目建议创建在：

```
museum-reservation-system/frontend
```

推荐目录结构：

```
frontend/  
├── package.json  
├── vite.config.js  
├── index.html  
├── src/  
│   ├── main.js  
│   ├── App.vue  
│   ├── router/  
│   │   └── index.js  
│   ├── api/  
│   │   ├── request.js  
│   │   ├── museum.js  
│   │   ├── visitor.js  
│   │   ├── reservation.js  
│   │   └── admin.js  
│   ├── layouts/  
│   │   ├── VisitorLayout.vue  
│   │   └── AdminLayout.vue  
│   ├── components/  
│   │   ├── AppNavbar.vue  
│   │   ├── AppFooter.vue  
│   │   ├── LoadingState.vue  
│   │   ├── EmptyState.vue  
│   │   ├── ErrorAlert.vue  
│   │   └── ConfirmDialog.vue  
│   ├── views/  
│   └── visitor/
```

```
| | | | |———| HomeView.vue
| | | | |———| VisitorLoginView.vue
| | | | |———| SlotListView.vue
| | | | |———| ReservationResultView.vue
| | | | |———| MyReservationsView.vue
| | | | |———| admin/
| | | | |———| AdminLoginView.vue
| | | | |———| DashboardView.vue
| | | | |———| MuseumManageView.vue
| | | | |———| NoticeManageView.vue
| | | | |———| ActivityManageView.vue
| | | | |———| SlotManageView.vue
| | | | |———| ReservationManageView.vue
| | | | |———| SystemStatusView.vue
| | | | |———| utils/
| | | | |———| storage.js
| | | | |———| validators.js
| | | | |———| format.js
| | | | |———| assets/
| | | | |———| styles/
| | | | |———| main.css
| | | | |
```

5. 路由设计

5.1 游客端路由

路由	页面组件	功能
/	HomeView.vue	场馆首页，展示场馆信息、公告、预约入口
/visitor/login	VisitorLoginView.vue	游客实名登录
/slots	SlotListView.vue	查询可预约时段并提交预约
/reservation/result	ReservationResultView.vue	展示预约结果、预约编号和二维码内容
/reservations/my	MyReservationsView.vue	查询我的预约记录

5.2 管理员端路由

路由	页面组件	功能
/admin/login	AdminLoginView.vue	管理员登录
/admin	DashboardView.vue	后台首页统计概览
/admin/museum	MuseumManageView.vue	场馆信息管理
/admin/notices	NoticeManageView.vue	公告通知管理
/admin/activities	ActivityManageView.vue	预约活动管理
/admin/slots	SlotManageView.vue	预约时段管理
/admin/reservations	ReservationManageView.vue	预约名单查询和导出
/admin/status	SystemStatusView.vue	系统状态查看

5.3 路由守卫设计

第一版不引入复杂权限系统，采用简单登录状态判断：

游客端：需要预约或查看我的预约时，检查 localStorage 中是否存在 visitorInfo。
管理员端：访问 /admin 相关页面时，检查 localStorage 中是否存在 adminInfo。

如果未登录：

游客跳转 /visitor/login
管理员跳转 /admin/login

6. Axios 请求封装设计

6.1 基础请求封装

文件：

src/api/request.js

职责：

- 设置 baseURL。
- 设置请求超时时间。
- 统一处理后端 ApiResponse 格式。
- 统一处理错误提示。

开发环境建议：

baseURL = http://127.0.0.1:8080

也可以在 .env.development 中配置：

VITE_API_BASE_URL=http://127.0.0.1:8080

后端统一响应格式：

```
{
  "success": true,
  "message": "操作成功",
  "data": {}
}
```

前端处理规则：

success = true：返回 data。
success = false：弹出 message，并抛出错误。
HTTP 500 或网络错误：提示系统繁忙或网络异常。

6.2 API 文件拆分

文件	职责
<code>museum.js</code>	场馆信息、公告、可预约时段
<code>visitor.js</code>	游客实名登录
<code>reservation.js</code>	提交预约、我的预约
<code>admin.js</code>	管理员登录和后台管理接口

7. 游客端页面设计

7.1 场馆首页 HomeView

功能：

- 展示场馆名称、地址、开放信息、参观须知。
- 展示启用公告列表。
- 提供“立即预约”按钮。
- 提供“我的预约”入口。

调用接口：

```
GET /api/museum/info
GET /api/notices
```

页面状态：

```
加载中
加载成功
加载失败
公告为空
```

7.2 游客实名登录 VisitorLoginView

功能：

- 输入姓名。
- 输入身份证号。
- 输入手机号。
- 调用后端实名登录接口。
- 登录成功后保存游客信息到 `localStorage`。

调用接口：

```
POST /api/visitor/login
```

表单校验：

```
姓名不能为空
身份证号必须为 18 位，最后一位可为数字或 X/x
```

手机号必须为 11 位大陆手机号格式

保存数据：

```
visitorInfo = {  
  visitorId,  
  name,  
  idCard,  
  phone  
}
```

7.3 预约时段选择 SlotListView

功能：

- 查询可预约时段。
- 支持按参观日期筛选。
- 展示日期、时段名称、开始时间、结束时间、总名额、已预约人数、剩余名额。
- 剩余名额为 0 时禁用预约按钮。
- 点击预约按钮后调用提交预约接口。

调用接口：

```
GET /api/slots  
GET /api/slots?visitDate=yyyy-MM-dd  
POST /api/reservations
```

预约前置条件：

游客必须先实名登录。
slotId 必须存在。
remaining > 0 时才允许点击预约按钮。

注意：

前端只能做体验层校验，最终是否可预约以后端返回结果为准。
后端已经实现事务、唯一约束和条件更新防止超额预约。

7.4 预约结果 ReservationResultView

功能：

- 展示预约成功或失败信息。
- 成功时展示预约编号、参观日期、时段、二维码内容。
- 提供返回首页、查看我的预约按钮。

数据来源：

```
POST /api/reservations 返回结果
```

二维码展示：

第一版可以直接展示 qrContent 文本。

增强版可以使用二维码组件生成二维码图片。

7.5 我的预约 MyReservationsView

功能：

- 根据当前游客身份证号查询预约记录。
- 也可以允许手动输入身份证号查询。
- 展示预约编号、日期、时段、状态、二维码内容、创建时间。

调用接口：

```
GET /api/reservations/my?idCard=身份证号
```

空状态：

```
暂无预约记录
```

8. 管理员端页面设计

8.1 管理员登录 AdminLoginView

功能：

- 输入用户名。
- 输入密码。
- 调用管理员登录接口。
- 登录成功后保存管理员信息到 localStorage。

调用接口：

```
POST /api/admin/login
```

保存数据：

```
adminInfo = {  
  adminId,  
  username,  
  role  
}
```

8.2 后台首页 DashboardView

功能：

- 展示游客数、预约数、总名额、已预约人数、剩余名额。
- 展示各时段名额统计。
- 提供按日期筛选统计。

调用接口：

```
GET /api/admin/summary
GET /api/admin/summary?visitDate=yyyy-MM-dd
```

可选增强：

使用 Chart.js 显示预约趋势或时段占用情况。

8.3 场馆信息管理 MuseumManageView

功能：

- 查询后台场馆信息。
- 编辑名称、地址、开放信息、参观须知、状态。
- 保存修改。

调用接口：

```
GET /api/admin/museum/info
PUT /api/admin/museum/info
```

8.4 公告通知管理 NoticeManageView

功能：

- 查询公告列表。
- 按启用状态和公告类型筛选。
- 新增公告。
- 编辑公告。
- 删除或禁用公告。

调用接口：

```
GET /api/admin/notices
GET /api/admin/notices?enabled=1
GET /api/admin/notices?type=RULE
POST /api/admin/notices
PUT /api/admin/notices/{id}
DELETE /api/admin/notices/{id}
```

公告类型：

```
NORMAL 普通通知
EXHIBITION 特殊展览
CLOSE 闭馆通知
RULE 预约规则
```

8.5 预约活动管理 ActivityManageView

功能：

- 查询预约活动。

- 按状态、日期范围筛选。
- 新增活动。
- 编辑活动。

调用接口：

```
GET /api/admin/activities
GET /api/admin/activities?status=OPEN
GET /api/admin/activities?startDate=yyyy-MM-dd&endDate=yyyy-MM-dd
POST /api/admin/activities
PUT /api/admin/activities/{id}
```

活动状态：

DRAFT 草稿
OPEN 启用
CLOSED 已关闭
DISABLED 禁用

8.6 预约时段管理 SlotManageView

功能：

- 查询预约时段。
- 按活动 ID 或参观日期筛选。
- 新增时段。
- 编辑时段。
- 启用或禁用时段。

调用接口：

```
GET /api/admin/slots
GET /api/admin/slots?activityId=1
GET /api/admin/slots?visitDate=yyyy-MM-dd
POST /api/admin/slots
PUT /api/admin/slots/{id}
```

注意：

同一活动下启用时段名额总和不能超过活动每日总名额。
已预约人数大于 0 的时段，后端不允许随意修改时间。

8.7 预约记录管理 ReservationManageView

功能：

- 查询预约记录。
- 按日期、时段、身份证号、状态筛选。
- 导出 CSV 预约名单。

调用接口：

GET /api/admin/reservations
GET /api/admin/reservations?visitDate=yyyy-MM-dd
GET /api/admin/reservations?slotId=1
GET /api/admin/reservations?idCard=身份证号
GET /api/admin/reservations?status=SUCCESS
GET /api/admin/reservations/export
GET /api/admin/reservations/export?visitDate=yyyy-MM-dd
GET /api/admin/reservations/export?slotId=1

CSV 导出说明：

后端返回 text/csv;charset=UTF-8 文件响应。
前端需要使用 window.open 或 axios blob 下载。

8.8 系统状态 SystemStatusView

功能：

- 展示系统状态。
- 展示数据库状态。
- 展示请求统计数量。
- 展示最近请求时间。

调用接口：

GET /api/admin/status
GET /api/health

说明：

当前 request_log 表如果暂无写入数据，则请求统计为 0、最近请求时间为 null，这是正常状态。

9. 页面与接口对应关系

页面	后端接口
场馆首页	GET /api/museum/info, GET /api/notices
游客登录页	POST /api/visitor/login
预约时段选择页	GET /api/slots, POST /api/reservations
预约结果页	使用 POST /api/reservations 返回数据
我的预约页	GET /api/reservations/my
管理员登录页	POST /api/admin/login
后台首页	GET /api/admin/summary, GET /api/admin/status
场馆信息管理页	GET /api/admin/museum/info, PUT /api/admin/museum/info
公告通知管理页	GET/POST/PUT/DELETE /api/admin/notices
预约活动管理页	GET/POST/PUT /api/admin/activities

预约时段管理页	GET/POST/PUT /api/admin/slots
预约记录管理页	GET /api/admin/reservations,GET /api/admin/reservations/export
系统状态页	GET /api/admin/status,GET /api/health

10. 前端状态管理设计

第一版不引入 Vuex 或 Pinia，使用简单本地状态和 localStorage。

10.1 游客状态

存储键：

visitorInfo

内容：

```
{
  "visitorId": 1,
  "name": "张三",
  "idCard": "110101199001010011",
  "phone": "13800138000"
}
```

使用场景：

- 提交预约时自动带入姓名、身份证号、手机号。
- 我的预约页自动使用身份证号查询。

10.2 管理员状态

存储键：

adminInfo

内容：

```
{
  "adminId": 1,
  "username": "admin",
  "role": "ADMIN"
}
```

使用场景：

- 控制后台路由访问。
- 后台导航栏展示当前管理员。

11. 前端表单校验设计

前端表单校验用于提升用户体验，但不能替代后端校验。

11.1 身份证号校验

规则：

18 位，前 17 位数字，最后一位数字或 X/x

正则：

`^[0-9]{17}[0-9Xx]$`

11.2 手机号校验

规则：

大陆 11 位手机号，1 开头，第二位 3-9

正则：

`^1[3-9][0-9]{9}$`

11.3 日期时间校验

预约活动表单需要校验：

bookingEnd > bookingStart
dailyCapacity > 0
personLimit > 0

预约时段表单需要校验：

endTime > startTime
totalCapacity > 0
enabled 只能是 0 或 1

12. 错误处理与提示设计

统一处理后端错误：

success=false 时显示 message。
网络错误时显示：网络异常，请稍后重试。
系统错误时显示：系统繁忙，请稍后重试。
表单错误时在对应输入框下方展示提示。

常见业务错误：

场景	前端提示来源
身份证号格式不正确	后端 message 或前端校验
手机号格式不正确	后端 message 或前端校验

预约时段不存在	后端 message
预约活动未开放	后端 message
预约尚未开始	后端 message
预约已经结束	后端 message
同一天重复预约	后端 message
预约名额已满	后端 message
管理员账号密码错误	后端 message

13. 前后端联调设计

13.1 后端启动

后端目录：

```
museum-reservation-system/backend
```

启动前需要配置数据库密码环境变量：

```
DB_PASSWORD=本地 MySQL 密码
```

启动命令：

```
java -jar target/museum-reservation-backend-0.0.1-SNAPSHOT.jar
```

或开发阶段使用：

```
mvn spring-boot:run
```

13.2 前端启动

前端目录：

```
museum-reservation-system/frontend
```

开发启动：

```
npm install
npm run dev
```

13.3 跨域

后端已配置 /api/** 跨域，前端可以直接请求：

```
http://127.0.0.1:8080/api/...
```

开发阶段也可以使用 Vite 代理：

/api -> http://127.0.0.1:8080

14. 前端测试计划

14.1 游客端测试

测试项	预期结果
打开首页	正常展示场馆信息和公告
游客实名登录	正确保存游客信息
身份证号错误	显示格式错误
手机号错误	显示格式错误
查询可预约时段	展示可预约时段和剩余名额
提交预约	返回预约编号和二维码内容
重复预约	显示同一天只能预约一次
查询我的预约	展示本人预约记录

14.2 管理员端测试

测试项	预期结果
管理员登录	登录成功进入后台
场馆信息修改	保存后游客首页展示新内容
公告新增	游客端公告可展示
公告禁用	游客端不再展示
活动新增	后台活动列表展示
时段新增	后台时段列表展示
时段名额超过活动总名额	后端返回失败
预约名单查询	展示预约记录
CSV 导出	下载预约名单文件
后台统计	显示统计数据
系统状态	显示 UP 和数据库状态

14.3 并发与一致性测试展示

前端本身不负责并发控制，但可以用于演示结果：

1. 打开预约时段页面查看剩余名额。
2. 使用接口压测或多窗口同时提交预约。
3. 回到页面刷新时段数据。
4. 确认 bookedCount 不超过 totalCapacity。
5. 确认同一身份证同一天不能重复预约。

15. 前端开发顺序

建议开发顺序：

1. 创建 Vue 3 + Vite 前端项目
2. 安装 Axios、Vue Router、Bootstrap
3. 配置基础目录结构
4. 配置 Axios 请求封装
5. 配置 Vue Router
6. 创建基础布局 VisitorLayout 和 AdminLayout
7. 实现场馆首页 HomeView
8. 实现游客实名登录 VisitorLoginView
9. 实现预约时段选择 SlotListView
10. 实现提交预约和预约结果 ReservationResultView
11. 实现我的预约 MyReservationsView
12. 实现管理员登录 AdminLoginView
13. 实现后台首页 DashboardView
14. 实现场馆信息管理 MuseumManageView
15. 实现公告通知管理 NoticeManageView
16. 实现预约活动管理 ActivityManageView
17. 实现预约时段管理 SlotManageView
18. 实现预约记录管理 ReservationManageView
19. 实现 CSV 导出
20. 实现系统状态 SystemStatusView
21. 做前后端联调测试
22. 做页面美化和演示数据整理
23. 准备部署和答辩演示

16. 当前还建议补充的内容

当前后端、数据库和接口文档已经比较完整。前端开发前，建议再补充或确认以下内容：

16.1 是否需要正式登录 Token

当前后端管理员登录和游客登录是课程大作业阶段的简单登录，不返回 JWT 或 Session Token。

前端第一版可以使用 `localStorage` 保存登录信息。

如果老师要求更严格权限控制，后续可补充：

JWT 登录认证
请求拦截器添加 Authorization Header
后端登录拦截器

但这不是当前主线必须项。

16.2 是否需要取消预约功能

当前接口文档和后端代码没有设计取消预约接口。

如果业务要求“游客可以取消预约”，需要新增：

```
PUT /api/reservations/{id}/cancel
```

并在事务中：

```
更新 reservation_record.status = CANCELLED  
减少 reservation_slot.booked_count
```

目前项目主线没有要求，可以暂不做。

16.3 是否需要扫码核验功能

当前系统只生成 `qrContent`，不包含扫码核验接口。

如果需要现场核验，可以后续扩展：

```
GET /api/reservations/verify?reservationNo=xxx
```

课程大作业第一版可以只展示二维码内容或预约凭证。

16.4 是否需要请求日志写入

当前 `request_log` 表存在，`GET /api/admin/status` 可以统计它，但后端暂未统一写入请求日志。

如果希望系统状态页展示真实请求量，可以后续补充：

```
预约接口成功/失败时写入 request_log  
或使用拦截器统一记录关键请求
```

当前不影响核心预约功能。

16.5 是否需要更强的图表展示

后台统计第一版可以用表格和数字卡片展示。

如果时间充足，可以增加：

```
Chart.js 柱状图  
预约名额占用饼图  
按日期预约趋势图
```

17. 与其他文档和后端实现核对结果

已对本文档与以下依据进行核对：

```
项目设计开发流程.md  
项目要求分析.md  
project-docs/需求拆分.md  
project-docs/系统设计文档初稿.md  
project-docs/数据库设计.md  
project-docs/后端接口设计.md
```


已完成的后端 Controller、DTO、Service 实现

核对结论：

1. 前端技术栈与项目定位一致，仍采用 Vue + Axios + Bootstrap。
2. 前端开发顺序与项目设计开发流程一致，采用模块完成后立即联调。
3. 游客端页面覆盖需求拆分和系统设计中的游客预约闭环。
4. 管理员端页面覆盖需求拆分和系统设计中的后台管理闭环。
5. 页面路由与页面职责没有发现冲突。
6. 页面与后端接口对应关系和后端接口设计一致。
7. 登录、场馆、公告、活动、时段、预约、统计、系统状态字段与已实现 DTO 基本一致。
8. 当前后端没有正式 Token、取消预约、扫码核验和请求日志自动写入，本文档已将这些内容列为可选增强，不作为第一版前端主线功能。

前端开发时应特别注意以下真实后端字段：

游客登录返回：visitorId, name, idCard, phone
管理员登录返回：adminId, username, role
场馆信息字段：id, name, address, openInfo, rules, status, updatedAt
公告类型：NORMAL, EXHIBITION, CLOSE, RULE
公告启用状态：enabled = 0 或 1
活动状态：DRAFT, OPEN, CLOSED, DISABLED
时段启用状态：enabled = 0 或 1
预约状态：SUCCESS, CANCELLED
CSV 导出接口返回 text/csv;charset=UTF-8, 不是 ApiResponse JSON
系统状态中 request_log 为空时，统计数量为 0, lastRequestTime 为 null

如果后续开发中发现文档与实际后端代码存在细微差异，处理优先级为：

1. 以已测试通过的后端接口实际返回为准。
2. 同步修正前端 API 封装和页面字段。
3. 必要时回写更新本文档或后端接口设计文档。

当前未发现会阻塞前端开发的文档矛盾。

18. 总结

前端开发应基于已经完成的后端 REST API 实现。当前建议优先完成游客端预约闭环，再完成管理员后台管理闭环。

核心闭环为：

游客首页 -> 游客实名登录 -> 查询可预约时段 -> 提交预约 -> 查看预约凭证 -> 我的预约记录

后台闭环为：

管理员登录 -> 后台统计 -> 场馆管理 -> 公告管理 -> 活动管理 -> 时段管理 -> 预约名单查询 -> CSV 导出 -> 系统状态查看

完成本文档后，可以进入前端项目搭建与页面开发阶段。